

ARC Investigator Data: Extraction and Cluster Construction

Overview

This report describes how raw Australian Research Council (ARC) grant data is converted into a cleaned set of unique investigator person-clusters suitable for bibliometric linkage. The process spans two scripts: `src/00_extract_arc.py`, which extracts structured records from raw JSON, and `src/01_prepare_arc.py`, which normalises names and constructs person-level clusters using probabilistic record linkage. The output is `arc_persons.parquet` — a table of 23,056 person-clusters derived from 65,087 grant-investigator rows.

1. Raw Data Extraction (`00_extract_arc.py`)

Input

The primary source is a single CSV file (`raw_json.csv`) containing 33,650 rows, each holding a JSON blob for one ARC grant. The JSON structure follows the ARC data portal API schema, with grant metadata and investigator lists nested under `data.attributes`.

Grant-Level Extraction

Grant metadata is extracted into `grants_flat.parquet`. Key fields include grant code, scheme, administering organisation, funding amounts, and grant summary text. A new field, `n_eligible_orgs`, counts the distinct universities named in `organisations-at-announcement` with roles of "Administering Organisation" or "Other Eligible Organisation". This distinguishes single-university grants (87.9% of in-scope grants) from multi-university collaborations — a distinction used later to guide automatic cluster merging.

Investigator Extraction

Each grant carries up to two investigator lists: `investigators-at-announcement` (the original team) and `investigators-current` (the final team). Both are processed and unioned by a `unique_id` key derived from the grant code and a normalised form of the investigator's name. Where the same `unique_id` appears in both lists, the current record's ORCID is preferred. This union strategy captures name-form changes over the life of a grant (e.g., "Chun Li" at announcement versus "Chun Guang Li" at completion) while preserving both forms as candidate evidence for deduplication.

The output is `investigators_raw.parquet`, containing 122,828 rows across all schemes. After filtering to in-scope schemes (DP, LP, FT, FL, FF, DE, DI) and eligible roles, 65,087 rows remain.

2. Name Normalisation — Phase 1 (`01_prepare_arc.py`)

Name Parsing

Phase 1 runs entirely in DuckDB SQL with Python UDFs. Each investigator row is processed by `arc_name_arrays()`, which uses the `HumanName` parser to split a concatenated full name into components.

Family names are stored as a single compound string (`family_name_main = max(family_names, key=len)`) to preserve compound surnames (van der Ent, de Almeida) that would be destroyed by tokenisation. First names are tokenised and stored as a set; a separate `first_initial` is derived from the longest token to avoid single-character initials when a full given name is available.

Diacritics are stripped by NFD normalisation followed by ASCII filtering. A separate Unicode-to-ASCII map handles non-decomposable characters ($\emptyset \rightarrow o$, $\ddot{u} \rightarrow ue$). Apostrophes are removed before tokenisation so that O'Brien and OBrien produce the same token.

FOR Code Normalisation

Each grant carries between one and three Field of Research (FOR) codes. Codes from the 2008 ANZSRC series are upgraded to 2020 equivalents using a concordance table, so that all clusters carry consistent ANZSRC 2020 codes. Field names are tokenised with domain stopwords removed; a concordance of 49 near-synonym FOR pairs ($\text{Jaccard} \geq 0.5$) allows token expansion at comparison time.

The output is `arc_investigators_prep.parquet` (65,087 rows), with blocking key columns `family_name_main` and `first_initial` ready for Splink.

3. Probabilistic Deduplication — Phase 2 (`01_prepare_arc.py`)

Splink Configuration

Phase 2 applies Splink's `dedupe_only` mode to identify rows that represent the same investigator. Blocking rules restrict comparisons to pairs sharing (`family_name_main`, `first_initial`) or a non-null ORCID. EM training uses the ORCID block to estimate match probabilities; the prior is set via `estimate_probability_two_random_records_match(recall=0.8)`. Clusters are formed at a threshold of 0.9 to favour precision over recall.

PROF

Comparisons include first name (exact, initial-match, and cross-matching for middle names), family name with term-frequency adjustment, full name key, institution intersection, FOR token overlap, and ORCID (seeded probabilities).

Post-Splink Cluster Construction

After Splink produces an initial clustering, a deterministic pipeline refines the result:

1. **ORCID merge:** clusters sharing an ORCID in the ARC source data are merged into the lexicographically smallest cluster ID.
2. **ORCID conflict split:** clusters holding two or more distinct ORCIDs are split; each ORCID subgroup becomes a new cluster.
3. **Multi-name split:** clusters spanning incompatible FOR divisions and co-investigator sets are split when the evidence is strong enough.

4. **Enriched ORCID promotion:** `00b_enrich_orcid.py` searches the ORCID public API for names not in the ARC source. High-confidence and single-AU-candidate results are promoted into the cluster's ORCID set. A blocklist prevents known false matches from being promoted.
5. **ERA FOR disambiguation:** low-confidence enrichment candidates with multiple AU results are resolved by matching the candidate ORCIDs' journal publication history (via the ERA 2023 journal list) against the cluster's ARC FOR codes. When exactly one candidate has a non-zero overlap, it is promoted.
6. **Manual overrides:** `config/manual_orcids.csv` (12 entries) injects verified ORCIDs; `config/manual_merges.csv` (11 entries) handles confirmed same-person pairs where automated steps failed.
7. **Same-grant coinvestigator merge:** the final deterministic step merges clusters that share a blocking key (`family_name_main`, `first_initial`) and appear together on the same single-university grant. Co-investigators on the same project at the same institution with the same name initial are treated as the same person regardless of Splink score. A conflict guard prevents merging when the clusters carry distinct non-empty ORCIDs.

Diagnostics

`01a_diagnose.py` applies a battery of checks after each run. Hard failures (MULTI_ORCID clusters, UNRESOLVED clusters, ORCID appearing in two clusters) must reach zero before downstream processing proceeds. Informational checks (B2: common names split across clusters; B3: compatible same-blocking-key pairs sharing co-investigators) guide further manual review.

Output

`arc_persons.parquet` contains 23,056 person-clusters. Each cluster aggregates full names, institution IDs, FOR codes, ORCIDs, and grant IDs from all contributing rows. Derived columns include `orcid_status` (HAS_ORCID / NO_ORCID / MULTI_ORCID), `orcid_for_codes` (ERA-derived field codes from ORCID works), `gap_candidates` (compatible unmerged same-name clusters for downstream review), and `reliability_tier` (1a–4u confidence classification). ORCID coverage is 73.1%.